

COMS 3110 Introduction to the Design and Analysis of Algorithms,

Homework 3

Mobina Amrollahi

Due: October 27th, 11:59pm

Late submission policy. An assignment that is submitted one day late will get a penalty of 10%. An assignment that is submitted two days late will get a penalty of 20%. Assignments submitted after two days will not be graded and will get no points. For example, if an assignment is due on Friday by midnight, a submission on Saturday will be penalized by 10%, and a submission on Sunday will be penalized by 20%. A submission on Monday will get no points.

Submission format. Homework solutions will have to be typed. You can use word, LaTeX, or any other type-setting tool to type your solution. Your submission file should be in pdf format. Do **NOT** submit a photocopy of handwritten homework except for diagrams that can be hand-drawn and scanned. We reserve the right **NOT** to grade homework that does not follow the formatting requirements. Name your submission file: `<Your-net-id>-3110-hw3.pdf`. For instance, if your netid is `asterix`, then your submission file will be named `asterix-3110-hw3.pdf`. Each student must hand in their own assignment. If you discussed the homework or solutions with others, a list of collaborators must be included with each submission. Each of the collaborators has to write the solutions in their own words (copies are not allowed).

General Requirements

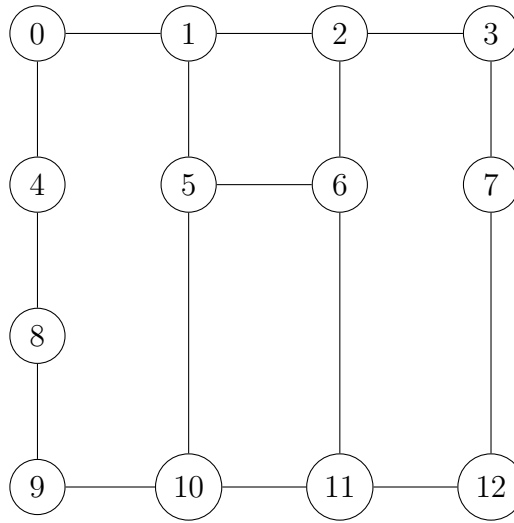
- When proofs are required, do your best to make them both clear and rigorous. Even when proofs are not required, you should justify your answers and explain your work.
- When asked to present a construction, you should show the correctness of the construction.

Some Useful (in)equalities

- $\sum_{i=1}^n i = \frac{n(n+1)}{2}$

- $\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$
 - $2^{\log_2 n} = n$, $a^{\log_b n} = n^{\log_b a}$, $n^{n/2} \leq n! \leq n^n$, $\log x^a = a \log x$
 - $\log(a \times b) = \log a + \log b$, $\log(a/b) = \log a - \log b$
 - $a + ar + ar^2 + \dots + ar^{n-1} = \frac{a(r^n-1)}{r-1}$
 - $1 + \frac{1}{2} + \frac{1}{2^2} + \dots + \frac{1}{2^n} = 2(1 - \frac{1}{2^{n+1}})$
 - $1 + 2 + 4 + \dots + 2^n = 2^{n+1} - 1$
-

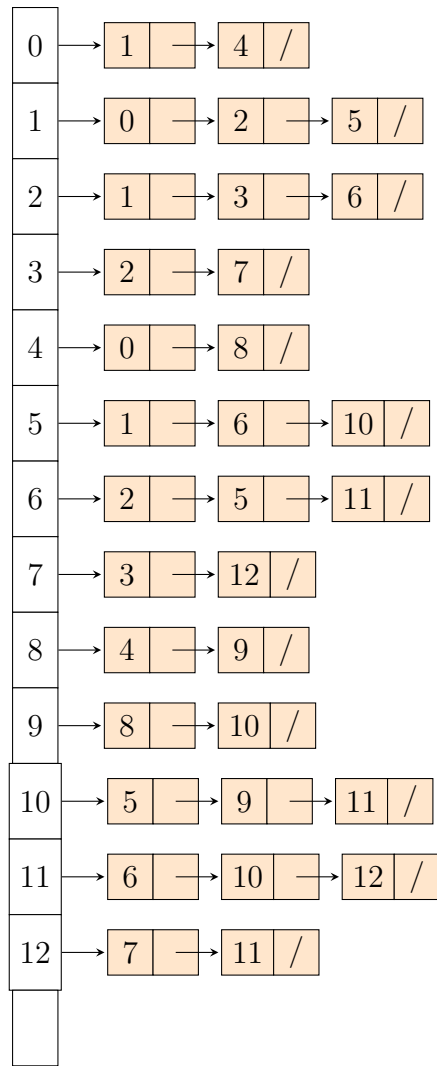
1. (10 pts) Consider the following graph G :



- (a) (2 pts) Write out the adjacency list representation for the graph G . At the list in each node, the vertices connected to it should be in increasing order of vertex number.

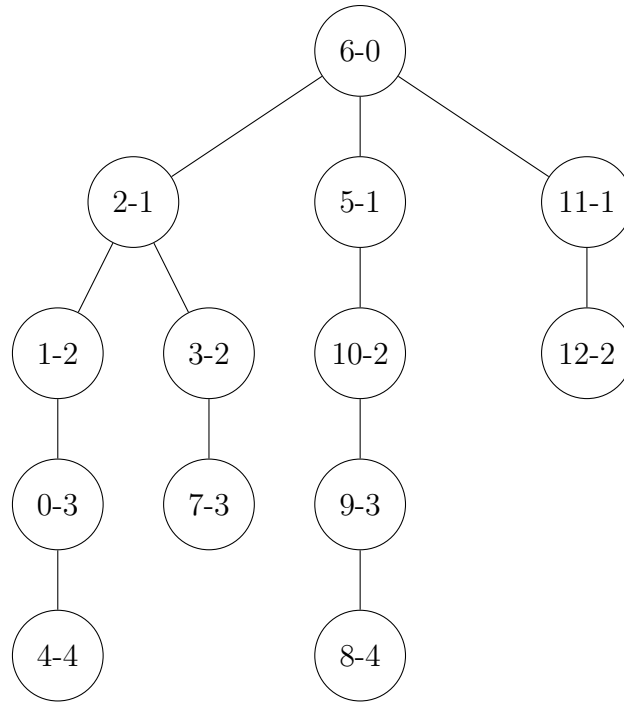
Answer

Please note that when I tried to draw the list with "tikzset" no matter what I did, the last empty slot, a white square, would always appear. That square is not part of my answer, and the adjacency list's last slot is number 12 and its neighbors.



- (b) **(6 pts)** Draw the BFS tree resulting from the BFS exploration of the above graph starting from vertex 6. Your exploration must consider the neighbors of any vertex in increasing order of vertex number. Show the layer information for each vertex in the BFS tree.

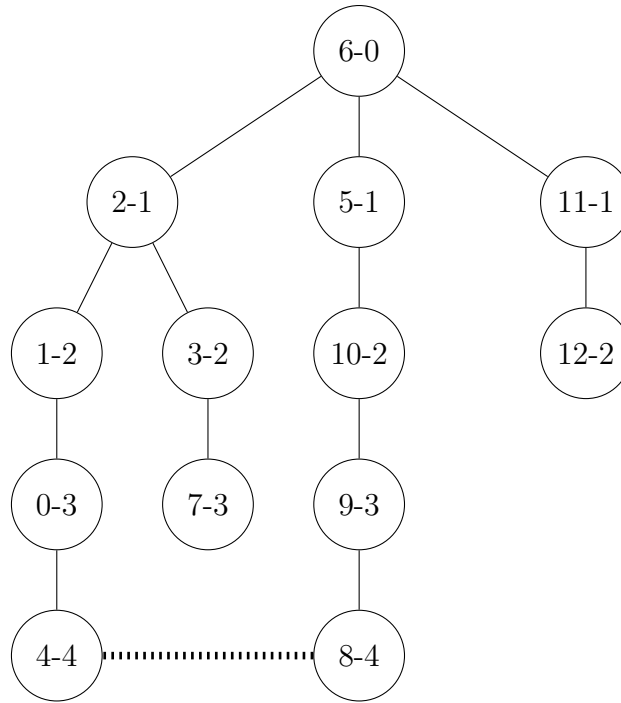
Answer



(c) **(2 pts)** Is the graph a bipartite graph? Justify your answer with an explanation.

Answer

No. This graph is not a bipartite graph since there's an edge between nodes 4 and 8, which are within the same layer. Having a connection between these two nodes results in an odd-length cycle in the graph, and odd-length cycles don't exist in a bipartite graph, i.e., a graph with an odd-length cycle cannot be bipartite. Explicitly, the cycle 4-8-9-10-5-6-2-1-0-4 is a cycle in our graph with length 9.



2. **(10 pts)** There are n -boxers. There are m -pairs among these boxers who are bitter rivals of each other. Write an algorithm that checks whether the boxers can be arranged in 2 groups such that rival boxers are in separate groups. If the check is successful, then your algorithm should output the two groups. Before developing your algorithm, give an outline of the steps in your solution strategy. Also, give the runtime analysis of your algorithm.

Answer

- (a) Description: We model the problem using a graph $\mathcal{G}(V, E)$, where each vertex in V represents a boxer, and two vertices are connected by an edge if their corresponding boxers are bitter rivals. Therefore:

$$|V| = n \quad \text{and} \quad |E| = m$$

The goal is to check whether this graph is bipartite. If the graph is bipartite, it means we can partition V into two sets A_0 and A_1 , such that all edges exist between one vertex in A_0 and one vertex in A_1 . This would imply that the boxers in each group are not bitter rivals. By performing a BFS, we can assign each vertex to a layer, with even-layered vertices going into A_0 and odd-layered vertices into A_1 . If an edge connects any two vertices in the same layer, the graph

is not bipartite, and no such partition exists, and the algorithm returns False. Finally, we will return A_0 and A_1 as an output.

(b) Pseudocode:

```

INPUT: Graph G, starting vertex v
Initialize A_0, A_1 = [], []

# Set explored flag to "False"
for all x in V:
    x.explored = false

# Obtain BFS(v) in graph G
v.explored = true
v.layer = 0
Add v to A_0
Initialize an empty queue Q
Add v to Q (to the tail)

while Q is not empty:
    Pop u from the head of Q

    for all neighbors w of u:
        if w.explored == false then:
            Add w to the tail of Q
            w.explored = true
            w.layer = u.layer + 1
            i = int(w.layer % 2) # Use 0 for A_0, 1 for A_1
            if i == 0:
                Add w to A_0
            else:
                Add w to A_1
        elif w.layer == u.layer:
            # w and u are in the same layer and connected
            Return False # The graph is not bipartite

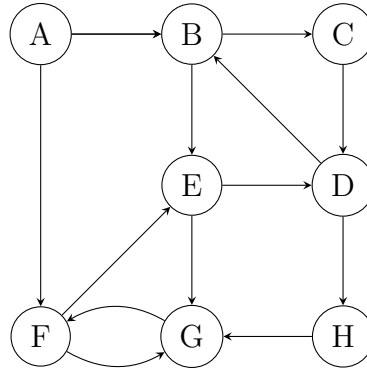
# Return the two partitions
Return A_0, A_1

```

(c) Run-time Analysis: The algorithm follows a breadth-first search (BFS) approach. Each vertex is visited once, and each edge is considered once when traversing neighbors. Thus, the time complexity of the algorithm is:

$$\text{Time Complexity} = \mathcal{O}(|V| + |E|)$$

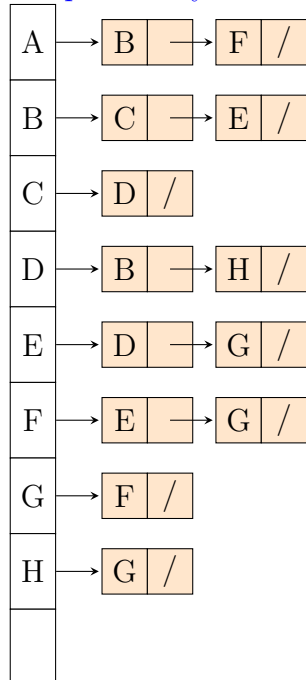
3. (10 pts) Consider the following graph H .



- (a) (3 pts) Write out the adjacency list representation for the graph H . At the list in each node, the vertices connected to it should be in alphabetic order (of vertex label).

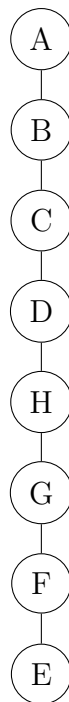
Answer

Please note that when I tried to draw the list with "tikzset" no matter what I did, the last empty slot, a white square, would always appear. That square is not part of my answer, and the adjacency list's last slot is "H" and its neighbors.



- (b) (7 pts) Draw the DFS tree resulting from the recursive DFS exploration starting at vertex A .

Answer



4. (10 pts) Consider an undirected graph with n vertices (assume n is even), no self loops, and each vertex has degree $\geq \frac{n}{2}$ (degree of a vertex is the number of edges incident at a vertex). Prove or disprove that all vertices in the graph belong to one connected component.

Answer

We will prove that this graph is one connected component where all vertices belong, and to do so, we will prove by contradiction.

Prove by Contradiction

For the sake of the contradiction, assume there are $k \geq 2$ connected components $\mathcal{G}_1\{V_1, E_1\}$, $\mathcal{G}_2\{V_2, E_2\}$, $\dots, \mathcal{G}_k\{V_k, E_k\}$ in this graph, where $|V_1| + |V_2| + \dots + |V_k| = |V| = n$ and each vertex belongs to exactly one. Since each vertex has a degree at least $\frac{n}{2}$, every vertex must be connected to at least $\frac{n}{2}$ other vertices. If a vertex belongs to a connected component, it must be connected to at least $\frac{n}{2}$ vertices within the same component, as there are no edges between different components. Therefore, each

connected component must have at least $\frac{n}{2} + 1$ vertices. Thus:

$$\begin{aligned} |V_1| &\geq \frac{n}{2} + 1 > \frac{n}{2}, \\ |V_2| &\geq \frac{n}{2} + 1 > \frac{n}{2}, \\ &\vdots \\ |V_k| &\geq \frac{n}{2} + 1 > \frac{n}{2} \end{aligned}$$

Summing these inequalities for k components, we get:

$$|V_1| + |V_2| + \cdots + |V_k| > k \times \frac{n}{2}.$$

Since $k \geq 2$, this implies:

$$|V_1| + |V_2| + \cdots + |V_k| > 2 \times \frac{n}{2} = n.$$

This contradicts the fact that the total number of vertices is n . Hence all vertices in the graph belong to one single connected component.

5. **(10 pts)** In a well-planned city, a road network connects intersections. If there is a road between two intersections, then one can go from one intersection to the connected intersection following the road connecting the two intersections. Each road has exactly the same length.

The city planning commission is trying to determine whether electric buses will be a viable option for public transportation in the city. Each bus is routed to travel along the *shortest* path from a starting intersection to a destination intersection. To reduce the range anxiety of the electric bus drivers, the commission wants to determine the longest such shortest path bus route in the city. Write an algorithm that finds the maximum such shortest path bus route.

Before developing your algorithm, give an outline of the steps in your solution strategy. Also, give the runtime analysis of your algorithm.

Answer

- (a) Description: We model the problem using a graph $\mathcal{G}(V, E)$, where each vertex in V represents intersections, and two vertices are connected by an edge if their corresponding intersection is connected by a road. Therefore:

$$|V| = \text{number of intersections} \quad \text{and} \quad |E| = \text{number of roads}$$

Since, if a road connects two intersections, that means we can go from one intersection to the other, this graph is an undirected graph. Hence, we need to find the longest shortest path between any two vertices. Thus, For each vertex v in V , we will Run BFS starting from v , which will find the shortest path to all other vertices, and we will keep track of the maximum distance found while running BFS(V). Finally, we will return the overall maximum distance found.

(b) Pseudocode:

```

INPUT: Graph G
Initialize max_path = 0

for all v in V:
    # Set explored flag to "False"
    for all x in V:
        x.explored = false

# Obtain BFS(v) in graph G
v.explored = true
v.layer = 0
Initialize an empty queue Q
Add v to Q (to the tail)
max_layer = 0

while Q is not empty:
    Pop u from the head of Q

    for all neighbors w of u:
        if w.explored == false then:
            Add w to the tail of Q
            w.explored = true
            w.layer = u.layer + 1
            max_layer = max(max_layer, w.layer)

    max_path = max(max_path, max_layer)

# Return the longest shortest path
Return max_path

```

(c) Run-time Analysis: In the initial for loop, which is iterating as much as the number of vertices, the dedicated explored flag for each vertex will be set to "False", which will take $\mathcal{O}(|V|)$, after that, "the breadth-first search (BFS) algorithm is obtained. In each iteration, the vertex is visited once, and each edge is considered

once when traversing neighbors. Thus, the time complexity of each iteration is:

$$\begin{aligned}
 \text{Time Complexity per Iteration} &= (\underbrace{\mathcal{O}(|V|)}_{\text{putting the explored flag as False}} \\
 &+ (\underbrace{\mathcal{O}(|V| + |E|)}_{\text{obtaining BFS for each vertex}} \\
 &= \mathcal{O}(2|V| + |E|) \\
 &\in \mathcal{O}(|V| + |E|)
 \end{aligned}$$

Thus, the time complexity of the algorithm is:

$$\text{Time Complexity} = \mathcal{O}(|V| \times (|V| + |E|)) = \mathcal{O}(|V|^2 + |V| \times |E|)$$

6. **(10 pts)** Given a directed graph and a vertex v , write an algorithm to check whether the vertex v can be reached from all vertices in the graph. Assume the graph is represented as an adjacency list. Before developing your algorithm, give an outline of the steps in your solution strategy. Give the runtime analysis of your algorithm.

- (a) Description: Instead of directly checking if vertex v can be reached from all other vertices, we can change this problem by reversing the graph. Let's define a reversed graph $\mathcal{G}_r(V, E)$ where:

- V remains the same as in our original graph
- For each edge (u, w) in the original graph, we add edge (w, u) in $\mathcal{G}_r$

This transformation changes our problem to whether we can reach all vertices from v in $\mathcal{G}_r$. If v can be reached from all vertices in the original graph, then v can reach all vertices in $\mathcal{G}_r$. To solve this, we can run $\text{BFS}(v)$ in $\mathcal{G}_r$ while counting the number of vertices visited during BFS. Finally, If the count equals $|V|$, then v can reach all vertices in $\mathcal{G}_r$, which means all vertices can reach v in the original graph. This works because BFS visits each reachable vertex exactly once.

Since we have the adjacency list, reversing the graph requires a specific approach. First, we create an empty adjacency list for $\mathcal{G}_r$ with $|V|$ empty lists (one for each vertex). Then, for each vertex u in $\mathcal{G}$'s adjacency list:

For each neighbor w in u 's list $\rightarrow$ Add u to the adjacency list in $\mathcal{G}_r$.

To maintain alphabetic/numeric ordering, for each vertex's adjacency list of $\mathcal{G}_r$, we process entries from the beginning of the adjacency list of $\mathcal{G}$. This ensures the reversed graph's adjacency lists maintain sorted order.

In this case, the algorithm contains two parts:

- i. Systematically constructing the adjacency list of $\mathcal{G}_r$ or reversing the graph $\mathcal{G}$

ii. Conducting $\text{BFS}(v)$ in the new graph

One last thing is to check and see whether v can be reached by itself in $\mathcal{G}$. This can be checked by seeing whether v has any neighbors in the adjacency list representing $\mathcal{G}_r$. If v has no outgoing edges in the reversed graph (which were incoming in the original graph), it can't be reachable.

(b) Pseudocode:

```
INPUT: adj # adjacency list of graph G
Initialize reversed_adj = {}
for v in adj:
    # Initialize empty lists for each vertex
    reversed_adj[v] = []

# Build a reversed graph
for u in adj:
    for w in adj[u]:
        reversed_adj[w].append(u)

# Check if v has outgoing edges in G_r
if reversed_adj[v] is empty:
    return False

# Set explored flag to "False"
explored = {}
for v in reversed_adj:
    explored[v] = False

# Obtain BFS(v) in graph G_r
count = 0
BFS(v):
    v.explored = true
    v.layer = 0
    Initialize an empty queue Q
    Add v to Q (to the tail)
    count += 1

    while Q is not empty:
        Pop u from the head of Q

        for all neighbors w of u:
            if w.explored == false then:
                Add w to the tail of Q
                w.explored = true
```

```

        w.layer = u.layer + 1
        count += 1

    if count == |V|:
        Return True

Return False

```

(c) Run-time Analysis: This algorithm has three separate parts:

- i. Reversing the graph: traversing lists of all vertices of $\mathcal{G}$, we can get $\mathcal{G}_r$. Thus the total time complexity of this part of the algorithm is $\mathcal{O}(|V| + |E|)$.
- ii. Check if v has outgoing edges in G_r : this can be done by checking the length of `reversed_adj[v]` and takes unit time.
- iii. Assigning the explored flag as False: the loop for this procedure is iterating as much as the number of vertices. For each iteration, the dedicated explored flag will be set to "False," which is considered an atomic operation. Thus, the total run time of the loop is: $\mathcal{O}(|V|)$
- iv. Conducting $\text{BFS}(v)$: The algorithm follows a breadth-first search (BFS) approach. Each vertex is visited once, and each edge is considered once when traversing neighbors. Thus, the time complexity of the algorithm is: $\mathcal{O}(|V| + |E|)$

Hence, the total runtime is:

$$\begin{aligned}
 \text{Time Complexity} &= \mathcal{O}(|V| + |E|) + \mathcal{O}(1) + \mathcal{O}(|V|) + \mathcal{O}(|V| + |E|) \\
 &= \mathcal{O}(3|V| + 2|E| + 1) \\
 &\in \mathcal{O}(|V| + |E|)
 \end{aligned}$$

List of Collaborators

Ryan Holt and I discussed our solutions to question 6 in this homework.

In question 6, he gave the idea of reversing the graph. In that case, the problem statement would change to finding out whether all the vertices in the graph can be reachable from vertex v . After sharing this idea, we separately wrote down the description, pseudocode, and algorithm and checked our final answers.